

Working remotely

Remote computing (and related issues)

- Logging on
- Copying files
- Retrieving data
- Compressing data
- Verifying data
- Preparing job scripts
- Submitting jobs

Logging onto a remote system

\$ `ssh` — establish a secure shell

- used to log into a remote machine and execute commands
- requires password authentication
- it's ***secure***, meaning that your login data are encrypted as they're transmitted through cyberspace
- UNSECURE connections: `telnet`, `rlogin`, and `ftp`
- can install OpenSSH on any Linux system
- can enable OpenSSH on any Mac

Logging onto a remote system

\$ `ssh` – establish a secure shell

\$ `ssh <username>@hpc-portal2.hpc.uark.edu`

... where `<username>` is:

- the name you requested, or
- in our case, your UARK username and password

File system on AHPCC

where you're at when you log in
\$ `/home/username` [on the "head node"]

Don't start running a bunch of analyses on the login node!

where to store your data
\$ `/storage/username`

where to run standard jobs from
\$ `$HOME` or `storage`

where to run jobs with high read/write
\$ `/scratch/$SLURM_JOB_ID`

1. copy input files to this directory
2. execute commands
3. copy output back to storage

Submitting a job to a remote queue

1. Get all your data in place on the remote system (see "File system on AHPCC")
2. Create a job script that includes:
 - all the information needed to run your job
 - all the commands you want to run, exactly as you would type them on the command line
3. Submit your job – as defined in your job script – to the queue

There are different queuing systems

1. TORQUE/PBS
2. SLURM (AHPCC)

Submitting and checking the job

what queues are available on the cluster?

\$ `sinfo -l`, or `qstat -q`

submit the job

\$ `sbatch <job_script.sh>`

when will my job start?

\$ `squeue -start -j <job id>`

where are programs installed

\$ `/share/apps`

what programs are installed as modules

\$ `module avail`

load a module

\$ `module load blast`

Submitting and checking the job

```
# check status of job
```

```
$ squeue -l -u <user_name>
```

```
# kill the job
```

```
$ scancel <Job ID>
```

```
# kill all my jobs
```

```
$ scancel -u <username>
```

```
# kill all my pending jobs
```

```
$ scancel -t PENDING -u <username>
```

Anatomy of a SLURM script

```
#!/bin/bash
```

```
#SBATCH --job-name=<insert job name>
```

```
#SBATCH --partition comp72  name of the queue
```

```
#SBATCH --nodes=1
```

```
#SBATCH --cpus-per-task=32
```

```
#SBATCH --time=72:00:00
```

```
#SBATCH -o %j.out
```

```
#SBATCH -e %j.err
```

```
#SBATCH --mail-type=ALL
```

```
#SBATCH --mail-user=aja@uark.edu
```

```
#SBATCH --qos comp
```

```
cd $SLURM_SUBMIT_DIR
```

```
# job command here
```

```
command 1
```

```
command 2
```

Anatomy of a SLURM script

```
#!/bin/bash
```

```
#SBATCH --job-name=<insert job name>
```

```
#SBATCH --partition comp72
```

```
#SBATCH --nodes=1  number of nodes
```

```
#SBATCH --cpus-per-task=32
```

```
#SBATCH --time=72:00:00
```

```
#SBATCH -o %j.out
```

```
#SBATCH -e %j.err
```

```
#SBATCH --mail-type=ALL
```

```
#SBATCH --mail-user=aja@uark.edu
```

```
#SBATCH --qos comp
```

```
cd $SLURM_SUBMIT_DIR
```

```
# job command here
```

```
command 1
```

```
command 2
```

Anatomy of a SLURM script

```
#!/bin/bash
```

```
#SBATCH --job-name=<insert job name>
```

```
#SBATCH --partition comp72
```

```
#SBATCH --nodes=1
```

```
#SBATCH --cpus-per-task=32
```

 number of cores
(take only what you need)

```
#SBATCH --time=72:00:00
```

```
#SBATCH -o %j.out
```

```
#SBATCH -e %j.err
```

```
#SBATCH --mail-type=ALL
```

```
#SBATCH --mail-user=aja@uark.edu
```

```
#SBATCH --qos comp
```

```
cd $SLURM_SUBMIT_DIR
```

```
# job command here
```

```
command 1
```

```
command 2
```

Anatomy of a SLURM script

```
#!/bin/bash
```

```
#SBATCH --job-name=<insert job name>
#SBATCH --partition comp72
#SBATCH --nodes=1
#SBATCH --cpus-per-task=32
#SBATCH --time=72:00:00  wall time
#SBATCH -o %j.out
#SBATCH -e %j.err
#SBATCH --mail-type=ALL
#SBATCH --mail-user=aja@uark.edu
#SBATCH --qos comp
```

```
cd $SLURM_SUBMIT_DIR
```

```
# job command here
```

```
command 1
```

```
command 2
```

Anatomy of a SLURM script

```
#!/bin/bash
```

```
#SBATCH --job-name=<insert job name>
```

```
#SBATCH --partition comp72
```

```
#SBATCH --nodes=1
```

```
#SBATCH --cpus-per-task=32
```

```
#SBATCH --time=72:00:00
```

```
#SBATCH -o %j.out  file names for STDOUT and STDERR;
```

```
#SBATCH -e %j.err this can be anything you want
```

```
#SBATCH --mail-type=ALL (%j is the job ID)
```

```
#SBATCH --mail-user=aja@uark.edu
```

```
#SBATCH --qos comp
```

```
cd $SLURM_SUBMIT_DIR
```

```
# job command here
```

```
command 1
```

```
command 2
```

Anatomy of a SLURM script

```
#!/bin/bash
```

```
#SBATCH --job-name=<insert job name>
```

```
#SBATCH --partition comp72
```

```
#SBATCH --nodes=1
```

```
#SBATCH --cpus-per-task=32
```

```
#SBATCH --time=72:00:00
```

```
#SBATCH -o %j.out
```

```
#SBATCH -e %j.err
```

```
#SBATCH --mail-type=ALL
```

 send me emails about the job

```
#SBATCH --mail-user=aja@uark.edu
```

```
#SBATCH --qos comp
```

```
cd $SLURM_SUBMIT_DIR
```

```
# job command here
```

```
command 1
```

```
command 2
```

Anatomy of a SLURM script

```
#!/bin/bash
```

```
#SBATCH --job-name=<insert job name>
```

```
#SBATCH --partition comp72
```

```
#SBATCH --nodes=1
```

```
#SBATCH --cpus-per-task=32
```

```
#SBATCH --time=72:00:00
```

```
#SBATCH -o %j.out
```

```
#SBATCH -e %j.err
```

```
#SBATCH --mail-type=ALL
```

```
#SBATCH --mail-user=aja@uark.edu
```

```
#SBATCH --qos comp
```

```
cd $SLURM_SUBMIT_DIR  cd into the working directory
```

```
# job command here
```

```
command 1
```

```
command 2
```

Anatomy of a SLURM script

```
#!/bin/bash
```

```
#SBATCH --job-name=<insert job name>
```

```
#SBATCH --partition comp72
```

```
#SBATCH --nodes=1
```

```
#SBATCH --cpus-per-task=32
```

```
#SBATCH --time=72:00:00
```

```
#SBATCH -o %j.out
```

```
#SBATCH -e %j.err
```

```
#SBATCH --mail-type=ALL
```

```
#SBATCH --mail-user=aja@uark.edu
```

```
#SBATCH --qos comp
```

```
cd $SLURM_SUBMIT_DIR
```

```
# job command here
```

```
command 1  command(s) to run
```

```
command 2
```

For jobs with lots of output, add this to the end of your SLURM script

```
# copy input files to the scratch disk before running the job  
rsync -arl * /scratch/$SLURM_JOB_ID/
```

```
# cd into the scratch folder to run the job  
cd /scratch/$SLURM_JOB_ID
```

```
# run the job  
command 1  
command 2
```

```
# cd back into the original folder when job has completed  
cd $SLURM_SUBMIT_DIR
```

```
# copy the output files back from the scratch disk  
rsync -arl /scratch/$SLURM_JOB_ID/* .
```

Copying files

Copying files *to* a remote computer

- \$ `scp` – Secure copy files between hosts on a network
 - Uses `ssh` for data transfer, so it offers the same security as `ssh`
 - Requires password authentication

Copying files between computers

Usage:

Basic structure: source file first, destination file second

\$ `scp [options] source destination`

Copying files between computers

Usage:

Copying between two computers

```
$ scp [options] path_to_file/filename \  
username@destination_host:my_directory/filename
```



IF usernames are not identical
on the 2 accounts, must specify
both username AND the host
name, separated by '@'

Copying files between computers

Usage:

Copying between two computers

```
$ scp [options] path_to_file/filename \  
username@destination_host:my_directory/filename
```



A colon separates the
destination host ID from the
destination filename ($\pm$ path)

Copying files between computers

Usage:

Copying between two computers

```
$ scp [options] path_to_file/filename \  
username@destination_host:my_directory/filename
```



Destination filename ($\pm$ path)

Copying files *to* a remote computer

Examples:

```
# Copy from current working directory to remote destination  
$ scp nad4.fasta aja@hpc-portal2.hpc.uark.edu:new_file.fasta
```

Copying directories *to* a remote computer

Examples:

Keep the same directory name

```
$ scp -r watermelon_nt aja@hpc-portal2.hpc.uark.edu:
```

Copy to a source directory

```
$ scp -r watermelon_nt aja@hpc-portal2.hpc.uark.edu:test_dir/copied_nt
```

Copy to a source directory, keep the same filename

```
$ scp -r watermelon_nt aja@hpc-portal2.hpc.uark.edu:/storage/aja
```

Copying *from* a remote computer

Examples:

```
# Copy from remote $HOME to current working directory
$ scp aja@hpc-portal2.hpc.uark.edu:nad4.fasta nad4_copied.fasta

# Copy from remote directory, keep the same filename
$ scp aja@hpc-portal2.hpc.uark.edu:blast_output/nad4.blastn .

# Copy to a source directory, keep the same filename
$ scp aja@hpc-portal2.hpc.uark.edu:nad4.fasta biol5153/watermelon_files/

# Copy directory from a remote computer
$ scp -r hpc-portal2.hpc.uark.edu:blast_output biol5153/watermelon_files/
```

Compress big files before moving them

Compressing files with `gzip`

`gzip` for compressing; `gunzip` for decompressing

- `gzip` can compress both files and STDIN
- writes to STDOUT by default
- operates on files in place by default (i.e., it doesn't create new files)

\$ `gzip [options]`

- `c` send output to STDOUT, allowing creation of duplicate files
- `d` decompression, rather than compression

Compressing files with `gzip`

Examples:

```
# compress STDIN
$ my_program.py illumina.fa.gz | gzip > out.fa.gz

# compress a file
$ gzip watermelon.fsa

# compress a file, creating a compressed copy
$ gzip -c watermelon.fsa > wat.fsa.gz

# unzip it
$ gunzip wat.fsa.gz
$ gzip -d wat.fsa.gz
```

Working with zipped files

You don't need to unzip a file to work with it

```
# view a zipped file
```

```
$ zless watermelon.fsa.gz
```

```
# view a zipped file
```

```
$ zcat watermelon.fsa.gz
```

```
# grep a zipped file
```

```
$ zgrep AATTGC watermelon.fsa.gz
```

Creating archives with `tar`

`tar` creates and manipulates tape archives

- allows for compression of entire directories
- preserves directory structure

\$ `tar [options] archive directory`

- `-c` create an archive
- `-x` extract an archive
- `-v` produce verbose output
- `-f <filename>` read the archive from or write it to the specified file
- `-t` list archive contents
- `-z` compress the archive with gzip

Creating archives with `tar`

Examples:

```
# archive and compress mt_genomes
$ cp -r ~/Desktop/watermelon_files/mt_genomes ~/Desktop
$ tar -czvf genomes.tgz mt_genomes/

# extract and decompress genomes.tgz
$ tar -xzvf genomes.tgz

# view contents of genomes.tgz
$ tar -tvf genomes.tgz
$ tar -tvf genomes.tgz

# extract 'ricinus.fasta' from genomes.tgz
$ tar -zxvf genomes.tgz watermelon_files/mt_genomes.ricinus.fasta
```

Make a single file of all the sequences in `watermelon_nt`, zip it, and copy it to `$HOME/mt_seqs` on razor.

Verifying data integrity

Verifying data integrity

- Download times can be very long for large files (e.g., raw sequence data)
- Network connections can drop out → partial loss of data
- Need to verify that your downloaded data is intact
- We use *checksums*, which are **highly compressed data summaries**
 - 40 characters in length
 - 16 characters, given in hexadecimal format (0–9, a–f) → 16^{40} possibilities
 - *checksum* algorithms are **deterministic** – the same dataset will always get the *checksum* value
- MD5 and SHA-1 are the two most common checksum algorithms

Verifying data integrity

The process:

1. Generate *checksum* values for original datasets → store them in a text file
2. User downloads data and text file with *checksum* values
3. User calculates *checksum* values on downloaded data
4. Compare both sets of values → any difference indicates corrupted data

* a single byte difference will give a different *checksum* value*

Verifying data integrity with `md5sum`

Examples:

```
# Calculate an MD5 hash on fruit.txt  
$ md5sum fruit.txt > fruit.md5sum
```

```
# Compare two MD5 hashes with Unix command diff  
$ diff fruit.md5sum fruit_original.md5sum
```

We can write a script later on to compare long sets of md5 hashes

Extras

Bigger, more sophisticated copies

Synchronizing directories with `rsync`

`rsync` for synchronizing directories

- only sends the *difference* between file versions
- can compress files during transfer
- the archive option preserves links, timestamps, permissions, ownership, and other attributes

\$ `rsync [options] source destination`

- `-a, --archive` preserve links, timestamps, etc.
- `-r, --recursive` recurse into directories
- `-v, --verbose` print lots of output about synchronization progress and status
- `-z, --compress` compress data as it is sent to its destination
- `-l, --links` preserve symbolic links
- `-e,` copy over SSH (note: easiest when you have a public SSH key)

Synchronizing directories with `rsync`

Examples:

```
# synchronize mt_genomes folder to your Desktop
$ cd ~/Desktop
$ rsync -av ~/Desktop/watermelon_files/mt_genomes .
$ ls -al ~/Desktop
```

```
# synchronize mt_genomes folder to your Desktop
$ cd ~/Desktop
$ rsync -av ~/Desktop/watermelon_files/mt_genomes/ .
$ ls -al ~/Desktop
```

What's the difference?

rsync: mind the '/'

```
# synchronize mt_genomes folder to your Desktop
# no trailing '/' copies the entire directory itself,
  including subdirectories
  $ cd ~/Desktop
  $ rsync -av ~/Desktop/watermelon_files/mt_genomes .

# synchronize mt_genomes folder to your Desktop
# trailing '/' means copy contents only - doesn't preserve
  directories
  $ rsync -av ~/Desktop/watermelon_files/mt_genomes/ .
```

Synchronizing directories with `rsync`

Examples:

```
# synchronize across machines
```

```
$ rsync -azv aja@razor.uark.edu:~/mysql .
```

```
# synchronize across machines - new destination directory
```

```
$ rsync -azv my_data aja@razor.uark.edu:~/my_data_new_name
```

Getting data

Downloading data with `wget`

`wget` for non-interactive file download from the web;
ftp and *http* are supported

```
$ wget [options] url
```

`-A, --accept` accept files matching this criterion

`-R, --reject` reject files matching this criterion

`-r, --recursive` recursive downloading (**warning**: very aggressive)

`--user=username` username for password authentication

`--ask-password` prompt for password

`--no-remove-listing` lists directory contents without downloading

Downloading data with `wget`

Examples:

```
# Download human chromosome 22 from UCSC Genome Browser
$ wget http://hgdownload.soe.ucsc.edu/goldenPath/hg19/chromosomes/chr22.fa.gz

# Download all the GFF files from your colleague's server
$ wget -accept "*.gff" --level 2 -r --no-directories --no-parent \
  ftp://server.university.edu/labsite/annotations/

# Download from a secure ftp site
$ wget ftp://<username>:<password>@192.169.8.66/folder

# Download from a secure ftp site
$ wget --recursive ftp://andy:run-dmc@192.169.8.66/CleanData
```

Downloading data with `curl`

`curl` ("see URL") for non-interactive file download

- more protocols than `wget`
- writes to STDOUT by default

```
$ curl [options] url
```

- o specifies output file

Downloading data with `curl`

Examples:

```
# Download a protein structure from RCSB's protein databank  
$ curl "http://www.rcsb.org/pdb/files/1ema.pdb"
```

```
# Write it to a file using -o option  
$ curl "http://www.rcsb.org/pdb/files/1ema.pdb" -o 1ema.pdb
```

```
# Download a set of files  
$ curl "http://www.rcsb.org/pdb/files/{1ema,1gfl,1g7k}.pdb"
```



Moving files with a GUI



- Shareware programs like [Fugu](#) and [FileZilla](#) provide a graphical front end for `ssh` and `scp`

<BEGIN> Do this on your own </BEGIN>

Authentication with SSH keys

- Repeatedly typing your password is a pain, and it takes time
- Not always safe – keyboard input can be monitored
- SSH keys are a safe and easy alternative
- Setting this up is pretty straightforward

NOTE: This can be necessary for authenticating GitHub login/push requests.

Authentication with SSH keys

1. On your ***local*** machine, check \$HOME/.ssh directory

```
total 192K
32K drwx----- 2 aja aja 32K Feb 4 22:32 ./
32K drwxr-xr-x 15 aja aja 32K Feb 4 22:31 ../
32K -rw----- 1 aja aja 1.4K Oct 24 2014 authorized_keys
32K -rw----- 1 aja aja 1.7K Oct 24 2014 id_rsa
32K -rw-r--r-- 1 aja aja 401 Oct 24 2014 id_rsa.pub
32K -rw-r--r-- 1 aja aja 9.2K Sep 19 08:49 known_hosts
```

 local computer
 remote host

2. If these files are missing, generate them with `ssh-keygen`:

```
$ ssh-keygen -b 2048 -t rsa
```

`-b`: number of bits in the key

`-t`: type of key to create

3. When prompted, store the keys in the default file and directory by hitting `<ENTER>`
4. Create and enter a password (`<ENTER>` for no password – this is okay)

Authentication with SSH keys

5. These files should now exist on your local machine

```
total 192K
32K drwx----- 2 aja aja 32K Feb 4 22:32 ./
32K drwxr-xr-x 15 aja aja 32K Feb 4 22:31 ../
32K -rw----- 1 aja aja 1.4K Oct 24 2014 authorized_keys
32K -rw----- 1 aja aja 1.7K Oct 24 2014 id_rsa
32K -rw-r--r-- 1 aja aja 401 Oct 24 2014 id_rsa.pub
32K -rw-r--r-- 1 aja aja 9.2K Sep 19 08:49 known_hosts
```

 local computer
 remote host

6. View and copy the ****public**** key

```
$ cat id_rsa.pub (then copy that line!)
```

7. SSH onto the remote machine

```
$ ssh username@hpc-portal2.hpc.uark.edu
```

8. Move into \$HOME/.ssh

```
$ cd .ssh
```

Authentication with SSH keys

9. Append the **public key** that you just copied from `id_rsa.pub` to the ``authorized_keys`` file

10. Make `authorized_keys` executable

```
$ chmod ugo+rx authorized_keys
```

11. Exit the remote host

```
$ exit
```

12. Log onto the remote host again

```
$ ssh username@hpc-portal2.hpc.uark.edu
```

13. If you created an SSH key password, enter it here if prompted

 local computer
 remote host

Authentication with SSH keys

14. If necessary, change file permissions in `.ssh/`

```
$ chmod go-w .ssh/
```

15. If you continue to be prompted for the SSH password:

```
# turn on SSH manager
```

```
$ eval ssh-agent
```

```
# tell SSH manager about your key
```

```
$ ssh-add
```

 local computer

 remote host

<END> Do this on your own </END>